JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class BreakIterator

java.lang.Object
    java.text.BreakIterator

All Implemented Interfaces:

Cloneable

```
public abstract class BreakIterator
extends Object
implements Cloneable
```

The `BreakIterator` class implements methods for finding the location of boundaries in text. Instances of `BreakIterator` maintain a current position and scan over text returning the index of characters where boundaries occur. Internally, `BreakIterator` scans text using a `CharacterIterator`, and is thus able to scan text held by any object implementing that protocol. A `StringCharacterIterator` is used to scan `String` objects passed to `setText`.

You use the factory methods provided by this class to create instances of various types of break iterators. In particular, use `getWordInstance`, `getLineInstance`, `getSentenceInstance`, and `getCharacterInstance` to create `BreakIterators` that perform word, line, sentence, and character boundary analysis respectively. A single `BreakIterator` can work only on one unit (word, line, sentence, and so on). You must use a different iterator for each unit boundary analysis you wish to perform.

Line boundary analysis determines where a text string can be broken when line-wrapping. The mechanism correctly handles punctuation and hyphenated words. Actual line breaking needs to also consider the available line width and is handled by higher-level software.

Sentence boundary analysis allows selection with correct interpretation of periods within numbers and abbreviations, and trailing punctuation marks such as quotation marks and parentheses.

Word boundary analysis is used by search and replace functions, as well as within text editing applications that allow the user to select words with a double click. Word selection provides correct interpretation of punctuation marks within and following words. Characters that are not part of a word, such as symbols or punctuation marks, have word-breaks on both sides.

Character boundary analysis allows users to interact with characters as they expect to, for example, when moving the cursor through a text string. Character boundary analysis provides correct navigation through character strings, regardless of how the character is stored. The boundaries returned may be those of supplementary characters, combining character sequences, or

ligature clusters. For example, an accented character might be stored as a base character and a diacritical mark. What users consider to be a character can differ between languages.

Implementation Requirements:

The default implementation of the character boundary analysis conforms to the Unicode Consortium's Extended Grapheme Cluster breaks. For more detail, refer to Grapheme Cluster Boundaries section in the Unicode Standard Annex #29.

Implementation Note:

The default implementations of `BreakIterator` will perform the equivalent of calling `setText("")` if the text hasn't been set by either `setText(String)` or `setText(CharacterIterator)` and a boundary searching operation is called by the `BreakIterator` instance. The `BreakIterator` instances returned by the factory methods of this class are intended for use with natural languages only, not for programming language text. It is however possible to define subclasses that tokenize a programming language.

Examples:

Creating and using text boundaries:

```
public static void main(String args[]) {
     if (args.length == 1) {
         String stringToExamine = args[0];
         //print each word in order
         BreakIterator boundary = BreakIterator.getWordInstance();
         boundary.setText(stringToExamine);
         printEachForward(boundary, stringToExamine);
         //print each sentence in reverse order
         boundary = BreakIterator.getSentenceInstance(Locale.US);
         boundary.setText(stringToExamine);
         printEachBackward(boundary, stringToExamine);
         printFirst(boundary, stringToExamine);
         printLast(boundary, stringToExamine);
     }
}
```

Print each element in order:

```
public static void printEachForward(BreakIterator boundary, String source)
    int start = boundary.first();
    for (int end = boundary.next();
         end != BreakIterator.DONE;
```

```
         start = end, end = boundary.next()) {
         System.out.println(source.substring(start,end));
    }
}
```

Print each element in reverse order:

```
public static void printEachBackward(BreakIterator boundary, String source)
    int end = boundary.last();
    for (int start = boundary.previous();
         start != BreakIterator.DONE;
         end = start, start = boundary.previous()) {
        System.out.println(source.substring(start,end));
    }
}
```

Print first element:

```
public static void printFirst(BreakIterator boundary, String source) {
    int start = boundary.first();
    int end = boundary.next();
    System.out.println(source.substring(start,end));
}
```

Print last element:

```
public static void printLast(BreakIterator boundary, String source) {
    int end = boundary.last();
    int start = boundary.previous();
    System.out.println(source.substring(start,end));
}
```

Print the element at a specified position:

```
public static void printAt(BreakIterator boundary, int pos, String source)
    int end = boundary.following(pos);
    int start = boundary.previous();
    System.out.println(source.substring(start,end));
}
```

Find the next word:

```
public static int nextWordStartAfter(int pos, String text) {
    BreakIterator wb = BreakIterator.getWordInstance();
    wb.setText(text);
    int last = wb.following(pos);
    int current = wb.next();
    while (current != BreakIterator.DONE) {
        for (int p = last; p < current; p++) {
            if (Character.isLetter(text.codePointAt(p)))
                return last;
        }
        last = current;
        current = wb.next();
    }
    return BreakIterator.DONE;
}
```

(The iterator returned by BreakIterator.getWordInstance() is unique in that the break positions it returns don't represent both the start and end of the thing being iterated over. That is, a sentence-break iterator returns breaks that each represent the end of one sentence and the beginning of the next. With the word-break iterator, the characters between two boundaries might be a word, or they might be the punctuation or whitespace between two words. The above code uses a simple heuristic to determine which boundary is the beginning of a word: If the characters between this boundary and the next boundary include at least one letter (this can be an alphabetical letter, a CJK ideograph, a Hangul syllable, a Kana character, etc.), then the text between this boundary and the next is a word; otherwise, it's the material between words.)

Since:

1.1

See Also:

`CharacterIterator`

Field Summary

Fields

Modifier and Type	Field	Description
`static final int`	**`DONE`**	DONE is returned by previous(), next(),

	next(int), preceding(int) and following(int) when either the first or last text boundary has been reached.

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	BreakIterator()	Constructor.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
Object	clone()	Create a copy of this iterator
abstract int	current()	Returns character index of the text boundary that was most recently returned by next(), next(int), previous(), first(), last(), following(int) or preceding(int).
abstract int	first()	Returns the first boundary.
abstract int	following(int offset)	Returns the first boundary following the specified character offset.
static Locale[]	getAvailableLocales()	Returns an array of all locales for which the get*Instance methods

		of this class can return localized instances.
static BreakIterator	**getCharacterInstance**()	Returns a new BreakIterator instance for character breaks for the default locale.
static BreakIterator	**getCharacterInstance** (Locale locale)	Returns a new BreakIterator instance for character breaks for the given locale.
static BreakIterator	**getLineInstance**()	Returns a new BreakIterator instance for line breaks for the default locale.
static BreakIterator	**getLineInstance** (Locale locale)	Returns a new BreakIterator instance for line breaks for the given locale.
static BreakIterator	**getSentenceInstance**()	Returns a new BreakIterator instance for sentence breaks for the default locale.
static BreakIterator	**getSentenceInstance** (Locale locale)	Returns a new BreakIterator instance for sentence breaks for the given locale.
abstract CharacterIterator	**getText**()	Get the text being scanned
static BreakIterator	**getWordInstance**()	Returns a new BreakIterator instance for word breaks for the default locale.
static BreakIterator	**getWordInstance** (Locale locale)	Returns a new BreakIterator instance

		for word breaks for the given locale.
boolean	**isBoundary**(int offset)	Returns true if the specified character offset is a text boundary.
abstract int	**last**()	Returns the last boundary.
abstract int	**next**()	Returns the boundary following the current boundary.
abstract int	**next**(int n)	Returns the nth boundary from the current boundary.
int	**preceding**(int offset)	Returns the last boundary preceding the specified character offset.
abstract int	**previous**()	Returns the boundary preceding the current boundary.
void	**setText**(String newText)	Set a new text string to be scanned.
abstract void	**setText**(CharacterIterator newTe	Set a new text for scanning.

Methods declared in class Object

equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Details

DONE

```
public static final int DONE
```

DONE is returned by previous(), next(), next(int), preceding(int) and following(int) when either the first or last text boundary has been reached.

See Also:

Constant Field Values

Constructor Details

BreakIterator

`protected BreakIterator()`

Constructor. BreakIterator is stateless and has no default behavior.

Method Details

clone

`public Object clone()`

Create a copy of this iterator

Overrides:

`clone` in class `Object`

Returns:

A copy of this

See Also:

`Cloneable`

first

`public abstract int first()`

Returns the first boundary. The iterator's current position is set to the first text boundary.

Returns:

The character index of the first text boundary.

last

`public abstract int last()`

Returns the last boundary. The iterator's current position is set to the last text boundary.

Returns:

The character index of the last text boundary.

next

```
public abstract int next(int n)
```

Returns the nth boundary from the current boundary. If either the first or last text boundary has been reached, it returns `BreakIterator.DONE` and the current position is set to either the first or last text boundary depending on which one is reached. Otherwise, the iterator's current position is set to the new boundary. For example, if the iterator's current position is the mth text boundary and three more boundaries exist from the current boundary to the last text boundary, the next(2) call will return m + 2. The new text position is set to the (m + 2)th text boundary. A next(4) call would return `BreakIterator.DONE` and the last text boundary would become the new text position.

Parameters:

`n` - which boundary to return. A value of 0 does nothing. Negative values move to previous boundaries and positive values move to later boundaries.

Returns:

The character index of the nth boundary from the current position or `BreakIterator.DONE` if either first or last text boundary has been reached.

next

```
public abstract int next()
```

Returns the boundary following the current boundary. If the current boundary is the last text boundary, it returns `BreakIterator.DONE` and the iterator's current position is unchanged. Otherwise, the iterator's current position is set to the boundary following the current boundary.

Returns:

The character index of the next text boundary or `BreakIterator.DONE` if the current boundary is the last text boundary. Equivalent to next(1).

See Also:

`next(int)`

previous

```
public abstract int previous()
```

Returns the boundary preceding the current boundary. If the current boundary is the first text boundary, it returns `BreakIterator.DONE` and the

iterator's current position is unchanged. Otherwise, the iterator's current position is set to the boundary preceding the current boundary.

Returns:

The character index of the previous text boundary or `BreakIterator.DONE` if the current boundary is the first text boundary.

following

```
public abstract int following(int offset)
```

Returns the first boundary following the specified character offset. If the specified offset is equal to the last text boundary, it returns `BreakIterator.DONE` and the iterator's current position is unchanged. Otherwise, the iterator's current position is set to the returned boundary. The value returned is always greater than the offset or the value `BreakIterator.DONE`.

Parameters:

`offset` - the character offset to begin scanning.

Returns:

The first boundary after the specified offset or `BreakIterator.DONE` if the last text boundary is passed in as the offset.

Throws:

`IllegalArgumentException` - if the specified offset is less than the first text boundary or greater than the last text boundary.

preceding

```
public int preceding(int offset)
```

Returns the last boundary preceding the specified character offset. If the specified offset is equal to the first text boundary, it returns `BreakIterator.DONE` and the iterator's current position is unchanged. Otherwise, the iterator's current position is set to the returned boundary. The value returned is always less than the offset or the value `BreakIterator.DONE`.

Parameters:

`offset` - the character offset to begin scanning.

Returns:

The last boundary before the specified offset or `BreakIterator.DONE` if the first text boundary is passed in as the offset.

Throws:

`IllegalArgumentException` - if the specified offset is less than the first text boundary or greater than the last text boundary.

Since:

1.2

isBoundary

`public boolean isBoundary(int offset)`

Returns true if the specified character offset is a text boundary.

Parameters:

`offset` - the character offset to check.

Returns:

`true` if "offset" is a boundary position, `false` otherwise.

Throws:

`IllegalArgumentException` - if the specified offset is less than the first text boundary or greater than the last text boundary.

Since:

1.2

current

`public abstract int current()`

Returns character index of the text boundary that was most recently returned by next(), next(int), previous(), first(), last(), following(int) or preceding(int). If any of these methods returns `BreakIterator.DONE` because either first or last text boundary has been reached, it returns the first or last text boundary depending on which one is reached.

Returns:

The text boundary returned from the above methods, first or last text boundary.

See Also:

`next()`, `next(int)`, `previous()`, `first()`, `last()`, `following(int)`, `preceding(int)`

getText

`public abstract CharacterIterator getText()`

Get the text being scanned

Returns:

the text being scanned

setText

`public void setText(String newText)`

Set a new text string to be scanned. The current scan position is reset to first().

Parameters:

`newText` - new text to scan.

setText

`public abstract void setText(CharacterIterator newText)`

Set a new text for scanning. The current scan position is reset to first().

Parameters:

`newText` - new text to scan.

getWordInstance

`public static BreakIterator getWordInstance()`

Returns a new `BreakIterator` instance for word breaks for the default locale.

Returns:

A break iterator for word breaks

getWordInstance

`public static BreakIterator getWordInstance(Locale locale)`

Returns a new `BreakIterator` instance for word breaks for the given locale.

Parameters:

`locale` - the desired locale

Returns:

A break iterator for word breaks

Throws:

`NullPointerException` - if `locale` is null

getLineInstance

`public static BreakIterator getLineInstance()`

Returns a new `BreakIterator` instance for line breaks for the default locale.

Returns:

A break iterator for line breaks

getLineInstance

`public static BreakIterator getLineInstance(Locale locale)`

Returns a new `BreakIterator` instance for line breaks for the given locale.

Parameters:

`locale` - the desired locale

Returns:

A break iterator for line breaks

Throws:

`NullPointerException` - if `locale` is null

getCharacterInstance

`public static BreakIterator getCharacterInstance()`

Returns a new `BreakIterator` instance for character breaks for the default locale.

Returns:

A break iterator for character breaks

getCharacterInstance

`public static BreakIterator getCharacterInstance(Locale locale)`

Returns a new `BreakIterator` instance for character breaks for the given locale.

Parameters:

`locale` - the desired locale

Returns:

A break iterator for character breaks

Throws:

`NullPointerException` - if `locale` is null

getSentenceInstance

`public static BreakIterator getSentenceInstance()`

Returns a new `BreakIterator` instance for sentence breaks for the default locale.

Returns:

A break iterator for sentence breaks

getSentenceInstance

```
public static BreakIterator getSentenceInstance(Locale locale)
```

Returns a new `BreakIterator` instance for sentence breaks for the given locale.

Parameters:

`locale` - the desired locale

Returns:

A break iterator for sentence breaks

Throws:

`NullPointerException` - if `locale` is null

getAvailableLocales

```
public static Locale[] getAvailableLocales()
```

Returns an array of all locales for which the `get*Instance` methods of this class can return localized instances. The returned array represents the union of locales supported by the Java runtime and by installed `BreakIteratorProvider` implementations. At a minimum, the returned array must contain a `Locale` instance equal to `Locale.ROOT` and a `Locale` instance equal to `Locale.US`.

Returns:

An array of locales for which localized `BreakIterator` instances are available.

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.